Learning RAG: The 20% That Matters

1 The Mental Model

RAG (Retrieval-Augmented Generation) connects an LLM to your own documents so it answers from your data instead of only its training. The idea is simple: before asking the LLM a question, you **retrieve** the most relevant chunks of your documents and paste them into the prompt as context. The model then **generates** an answer grounded in that context. This avoids fine-tuning and lets you update knowledge by just changing the documents.

The pipeline has two phases:

- **Indexing (offline, once):** load → chunk → embed → store in a vector database.
- **Querying (online, per question):** embed query → retrieve top chunks → (rerank) → stuff into prompt → generate.

2 The Single Most Important Lesson

Retrieval quality matters more than which LLM you use. The dominant industry finding is that the large majority of RAG failures come from the ingestion and chunking layer, not the model: if the retriever hands the LLM the wrong passage, the model writes a confident, articulate, wrong answer. Spend your effort on chunking and retrieval first.

3 Step 1 — Load and Chunk

Documents are too big to embed whole, so you split them into chunks. The recommended default is **recursive character splitting**, which tries to break on paragraphs, then sentences, then words — keeping related text together.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,        # characters per chunk (tune this!)
    chunk_overlap=120,     # overlap so context isn't cut mid-thought
)
chunks = splitter.split_text(long_document_text)
```

Chunking rules of thumb:

- Too small → the embedding has almost nothing to represent.
- Too large → multiple topics dilute the vector, hurting precision.
- Always use some **overlap** so a fact split across a boundary isn't lost.

- Treat `chunk_size` as a tunable parameter, not a fixed default.
- Clean input matters: split on markdown headers/structure, not raw HTML.

4 Step 2 — Embeddings

An embedding turns text into a vector (a list of floats) where similar meanings are near each other. You embed every chunk once, and embed each query at search time. Same model for both.

```python
# Option A: local, free, via sentence-transformers (Hugging Face)
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")  # small, fast, 384-dim
vectors = model.encode(chunks)   # shape: (num_chunks, 384)

# Option B: hosted API (e.g. OpenAI)
# vec = client.embeddings.create(model="text-embedding-3-small", input=
    text)
```

Key shape fact: an embedding model has a fixed output dimension (e.g. 384, 768, 1536). Every vector in your store must use the same model and dimension. Queries are short and documents long — some models embed them slightly differently, which is normal.

5 Step 3 — Vector Database

The vector DB stores chunk vectors and finds the nearest ones to a query vector. Chroma is the easiest to start with; Pinecone, Qdrant, Weaviate, Milvus, and pgvector are common in production.

```python
import chromadb

client = chromadb.Client()
collection = client.create_collection("docs")

# Add chunks (Chroma can embed for you, or pass your own vectors)
collection.add(
    documents=chunks,
    ids=[f"chunk-{i}" for i in range(len(chunks))],
    metadatas=[{"source": "manual.pdf"} for _ in chunks],  # for filtering
)

# Query
results = collection.query(query_texts=["How do I reset my password?"],
    n_results=5)
top_chunks = results["documents"][0]
```

Two technical knobs to know: the **distance metric** (cosine similarity is the usual choice) and the **ANN index** (HNSW is the common default). Be consistent across your collection.

6 Step 4 — Retrieve and Generate

Assemble the retrieved chunks into a prompt and ask the LLM, instructing it to answer only from the context.

```
context = "\n\n".join(top_chunks)

prompt = f"""Answer the question using ONLY the context below.
If the answer isn't in the context, say you don't know.

Context:
{context}

Question: {user_question}
"""

# Send to any LLM API (see the API-calls document)
answer = call_llm(prompt)
```

Prompt discipline: explicitly telling the model to use only the context and to admit when it doesn't know is the cheapest, highest-impact way to reduce hallucination.

7 The Two-Stage Upgrade: Retrieve then Rerank

This is now standard practice and the biggest quality win after good chunking. Plain embedding search is fast but lossy. So:

1. **Stage 1 (retrieve):** pull the top 50–100 candidates from the vector index using fast embedding (bi-encoder) search.
2. **Stage 2 (rerank):** score each (query, chunk) pair with a slower but more accurate cross-encoder reranker, then keep only the top 3–10 for the prompt.

```
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
pairs = [(query, chunk) for chunk in candidate_chunks]
scores = reranker.predict(pairs)              # higher = more relevant

ranked = sorted(zip(candidate_chunks, scores), key=lambda x: -x[1])
final_chunks = [c for c, s in ranked[:5]]    # top 5 to the LLM
```

Hosted rerankers exist too (Cohere Rerank, Voyage, Jina, BGE families), usually behind a simple HTTP call.

8 Hybrid Search (Dense + Sparse)

Embedding (dense) search catches *meaning*; keyword (sparse, BM25) search catches *exact terms*, names, and acronyms. Combining both before reranking catches cases each alone would miss — especially useful for technical corpora full of specific identifiers.

9 The Small-to-Big Pattern

A widely used trick: index **small** chunks for precise retrieval, but return the surrounding **larger parent** chunk to the LLM for richer context. You get accurate matching plus enough surrounding text for a good answer.

10 Putting It Together (Minimal RAG)

```python
# --- INDEXING (run once) ---
chunks = splitter.split_text(load_documents())
collection.add(documents=chunks, ids=[f"c{i}" for i in range(len(chunks))])

# --- QUERYING (per question) ---
def rag(question: str) -> str:
    hits = collection.query(query_texts=[question], n_results=50)["documents"][0]
    pairs = [(question, c) for c in hits]
    scores = reranker.predict(pairs)
    top = [c for c, s in sorted(zip(hits, scores), key=lambda x: -x[1])[:5]]
    context = "\n\n".join(top)
    return call_llm(f"Use only this context:\n{context}\n\nQ: {question}")
```

This pairs naturally with FastAPI (wrap `rag()` in a `POST /ask` endpoint) and Docker (ship the whole thing in a container).

11 Evaluation (Don't Skip This)

You cannot improve what you don't measure. Evaluate the two halves separately:

- **Retriever:** did it fetch the right chunks? Metrics like context precision (fraction of retrieved chunks that were actually relevant) and recall.
- **Generator:** did the answer faithfully use the retrieved context, without making things up?

The RAGAS library is a common tool for this. Without evaluation you can't tell whether a change to chunking, embedding, or retrieval actually helped.

12 Common Failure Modes and Fixes

- **Right answer buried in noise** → reduce chunk size or use hierarchical chunking.
- **Misses obvious matches** → add hybrid (BM25) search; check the embedding model fits your domain.
- **Stale answers** → your re-indexing pipeline is out of date; refresh the index.
- **Hallucination** → tighten the prompt ("answer only from context"); add reranking to send cleaner context.
- **Bloated, slow context** → keep assembled context modest (well under the model's limit); be stricter with the rerank cutoff.

13 The Checklist for Any RAG System

1. Load and clean documents into good structure.

2. Chunk sensibly (recursive split, overlap, tuned size).
3. Embed with one consistent model; store vectors + metadata.
4. Retrieve a wide candidate set, then rerank down to a few.
5. Build a disciplined prompt; generate.
6. Evaluate retriever and generator separately; iterate.

14 Practice Task

Build a tiny RAG over a few text files of your own:

1. Split them with `RecursiveCharacterTextSplitter` (size 800, overlap 120).
2. Embed with `all-MiniLM-L6-v2` and store in a Chroma collection.
3. Write a `rag(question)` function: retrieve top 20, rerank to top 5, build the prompt.
4. Ask a question whose answer is in your files; then ask one that isn't and confirm it says "I don't know."
5. Bonus: wrap it in a FastAPI `POST /ask` endpoint.